

T_{25}	T_{26}
read(B)	read(B) $B := B - 50$ read(A) $A := A + 50$
read(A) <validate> display($A + B$)	<validate> write(B) write(A)

Figure 18.19 Schedule 6, a schedule produced by using validation.

This validation scheme is called the **optimistic concurrency-control** scheme since transactions execute optimistically, assuming they will be able to finish execution and validate at the end. In contrast, locking and timestamp ordering are pessimistic in that they force a wait or a rollback whenever a conflict is detected, even though there is a chance that the schedule may be conflict serializable.

It is possible to use $TS(T_i) = StartTS(T_i)$ instead of $ValidationTS(T_i)$ without affecting serializability. However, doing so may result in a transaction T_i entering the validation phase before a transaction T_j that has $TS(T_j) < TS(T_i)$. Then, the validation of T_i would have to wait for T_j to complete, so its read and write sets are completely known. Using $ValidationTS$ avoids this problem.

18.7 Multiversion Schemes

The concurrency-control schemes discussed thus far ensure serializability by either delaying an operation or aborting the transaction that issued the operation. For example, a read operation may be delayed because the appropriate value has not been written yet; or it may be rejected (that is, the issuing transaction must be aborted) because the value that it was supposed to read has already been overwritten. These difficulties could be avoided if old copies of each data item were kept in a system.

In **multiversion concurrency-control** schemes, each $write(Q)$ operation creates a new **version** of Q . When a transaction issues a $read(Q)$ operation, the concurrency-control manager selects one of the versions of Q to be read. The concurrency-control scheme must ensure that the version to be read is selected in a manner that ensures serializability. It is also crucial, for performance reasons, that a transaction be able to determine easily and quickly which version of the data item should be read.

18.7.1 Multiversion Timestamp Ordering

The timestamp-ordering protocol can be extended to a multiversion protocol. With each transaction T_i in the system, we associate a unique static timestamp, denoted by $TS(T_i)$. The database system assigns this timestamp before the transaction starts execution, as described in Section 18.5.

With each data item Q , a sequence of versions $\langle Q_1, Q_2, \dots, Q_m \rangle$ is associated. Each version Q_k contains three data fields:

1. **Content** is the value of version Q_k .
2. **W-timestamp**(Q_k) is the timestamp of the transaction that created version Q_k .
3. **R-timestamp**(Q_k) is the largest timestamp of any transaction that successfully read version Q_k .

A transaction—say, T_i —creates a new version Q_k of data item Q by issuing a **write**(Q) operation. The content field of the version holds the value written by T_i . The system initializes the W-timestamp and R-timestamp to $TS(T_i)$. It updates the R-timestamp value of Q_k whenever a transaction T_j reads the content of Q_k and $R\text{-timestamp}(Q_k) < TS(T_j)$.

The **multiversion timestamp-ordering scheme** presented next ensures serializability. The scheme operates as follows: Suppose that transaction T_i issues a **read**(Q) or **write**(Q) operation. Let Q_k denote the version of Q whose write timestamp is the largest write timestamp less than or equal to $TS(T_i)$.

1. If transaction T_i issues a **read**(Q), then the value returned is the content of version Q_k .
2. If transaction T_i issues **write**(Q), and if $TS(T_i) < R\text{-timestamp}(Q_k)$, then the system rolls back transaction T_i . On the other hand, if $TS(T_i) = W\text{-timestamp}(Q_k)$, the system overwrites the contents of Q_k ; otherwise (if $TS(T_i) > R\text{-timestamp}(Q_k)$), it creates a new version of Q .

The justification for rule 1 is clear. A transaction reads the most recent version that comes before it in time. The second rule forces a transaction to abort if it is “too late” in doing a write. More precisely, if T_i attempts to write a version that some other transaction would have read, then we cannot allow that write to succeed.

The **valid interval** of a version Q_i of Q with W-timestamp t is defined as follows: if Q_i is the latest version of Q , the interval is $[t, \infty]$; otherwise let the next version of Q have timestamp s ; then the valid interval is $[t, s)$. You can easily verify that reads by a transaction with timestamp t_i return the content of the version whose valid interval contains t_i .

Versions that are no longer needed are removed according to the following rule: Suppose that there are two versions, Q_k and Q_j , of a data item, and that both versions

have a W-timestamp less than the timestamp of the oldest transaction in the system. Then, the older of the two versions Q_k and Q_j will not be used again, and can be deleted.

The multiversion timestamp-ordering scheme has the desirable property that a read request never fails and is never made to wait. In typical database systems, where reading is a more frequent operation than is writing, this advantage may be of major practical significance.

The scheme, however, suffers from two undesirable properties. First, the reading of a data item also requires the updating of the R-timestamp field, resulting in two potential disk accesses, rather than one. Second, the conflicts between transactions are resolved through rollbacks, rather than through waits. This alternative may be expensive. Section 18.7.2 describes an algorithm to alleviate this problem.

This multiversion timestamp-ordering scheme does not ensure recoverability and cascadelessness. It can be extended in the same manner as the basic timestamp-ordering scheme to make it recoverable and cascadeless.

18.7.2 Multiversion Two-Phase Locking

The **multiversion two-phase locking protocol** attempts to combine the advantages of multiversion concurrency control with the advantages of two-phase locking. This protocol differentiates between **read-only transactions** and **update transactions**.

Update transactions perform rigorous two-phase locking; that is, they hold all locks up to the end of the transaction. Thus, they can be serialized according to their commit order. Each version of a data item has a single timestamp. The timestamp in this case is not a real clock-based timestamp, but rather is a counter, which we will call the **ts-counter**, that is incremented during commit processing.

The database system assigns read-only transactions a timestamp by reading the current value of **ts-counter** before they start execution; they follow the multiversion timestamp-ordering protocol for performing reads. Thus, when a read-only transaction T_i issues a $\text{read}(Q)$, the value returned is the contents of the version whose timestamp is the largest timestamp less than or equal to $\text{TS}(T_i)$.

When an update transaction reads an item, it gets a shared lock on the item and reads the latest version of that item. When an update transaction wants to write an item, it first gets an exclusive lock on the item and then creates a new version of the data item. The write is performed on the new version, and the timestamp of the new version is initially set to a value ∞ , a value greater than that of any possible timestamp.

When the update transaction T_i completes its actions, it carries out commit processing; only one update transaction is allowed to perform commit processing at a time. First, T_i sets the timestamp on every version it has created to 1 more than the value of **ts-counter**; then, T_i increments **ts-counter** by 1, and commits.

Read-only transactions see the old value of **ts-counter** until T_i has successfully committed. As a result, read-only transactions that start after T_i commits will see the values updated by T_i , whereas those that start before T_i commits will see the value before the updates by T_i . In either case, read-only transactions never need to wait for

Note 18.2 MULTIVERSIONING AND DATABASE IMPLEMENTATION

Consider a database system that implements a primary key constraint by ensuring that only one tuple exists for any value of the primary key attribute. The creation of a second version of the record with the same primary key would appear to be a violation of the primary key constraint. However, it is logically not a violation, since the two versions do not coexist at any time in the database. Therefore, primary constraint enforcement must be modified to allow multiple records with the same primary key, as long as they are different versions of the same record.

Next, consider the issue of deletion of tuples. This can be implemented by creating a new version of the tuple, with timestamps created as usual, but with a special marker denoting that the tuple has been deleted. Transactions that read such a tuple simply skip it, since it has been deleted.

Further, consider the issue of enforcing foreign-key dependencies. Consider the case of a relation r whose attribute $r.B$ is a foreign-key referencing attribute $s.B$ of relation s . In general, deletion of a tuple t_s in s or update of a primary key attribute of tuple t_s in s causes a foreign-key violation if there is an r tuple t_r such that $t_r.B = t_s.B$. With multiversioning, if the timestamp of the transaction performing the deletion/update is ts_i , the corresponding condition for violation is the existence of such a tuple version t_r , with the additional condition that the valid interval of t_r contains ts_i .

Finally, consider the case of an index on attribute $r.B$ of relation r . If there are multiple versions of a record t_i with the same value for B , the index could point to the latest version of the record, and the latest version could have pointers to earlier versions. However, if an update was made to attribute $t_i.B$, the index would need to contain separate entries for different versions of record t_i ; one entry for the old value of $t_i.B$ and another for the new value of $t_i.B$. When old versions of a record are deleted, any entry in the index for the old version must also be deleted.

locks. Multiversion two-phase locking also ensures that schedules are recoverable and cascadeless.

Versions are deleted in a manner like that of multiversion timestamp ordering. Suppose there are two versions, Q_k and Q_j , of a data item, and that both versions have a timestamp less than or equal to the timestamp of the oldest read-only transaction in the system. Then, the older of the two versions Q_k and Q_j will not be used again and it can be deleted.

18.8 Snapshot Isolation

Snapshot isolation is a particular type of concurrency-control scheme that has gained wide acceptance in commercial and open-source systems, including Oracle,

PostgreSQL, and SQL Server. We introduced snapshot isolation in Section 17.9.3. Here, we take a more detailed look into how it works.

Conceptually, snapshot isolation involves giving a transaction a “snapshot” of the database at the time when it begins its execution. It then operates on that snapshot in complete isolation from concurrent transactions. The data values in the snapshot consist only of values written by committed transactions. This isolation is ideal for read-only transactions since they never wait and are never aborted by the concurrency manager.

Transactions that update the database potentially have conflicts with other transactions that update the database. Updates performed by a transaction must be validated before the transaction is allowed to commit. We describe how validation is performed, later in this section. Updates are kept in the transaction’s private workspace until the transaction is validated, at which point the updates are written to the database.

When a transaction T is allowed to commit, the transition of T to the committed state and the writing of all of the updates made by T to the database must be conceptually done as an atomic action so that any snapshot created for another transaction either includes all updates by transaction T or none of them.

18.8.1 Multiversioning in Snapshot Isolation

To implement snapshot isolation, transactions are given two timestamps. The first timestamp, $StartTS(T_i)$, is the time at which transaction T_i started. The second timestamp, $CommitTS(T_i)$ is the time when the transaction T_i requested validation.

Note that timestamps can be wall clock time, as long as no two transactions are given the same timestamp, but they are usually assigned from a counter that is incremented every time a transaction enters its validation phase.

Snapshot isolation is based on multiversioning, and each transaction that updates a data item creates a version of the data item. Versions have only one timestamp, which is the write timestamp, indicating when the version was created. The timestamp of a version created by transaction T_i is set to $CommitTS(T_i)$. (Since updates to the database are also only made after validation of the transaction T_i , $CommitTS(T_i)$ is available when a version is created.)²

When a transaction T_i reads a data item, the latest version of the data item whose timestamp is $\leq StartTS(T_i)$ is returned to T_i . Thus, T_i does not see the updates of any transactions that committed after T_i started, while it does see the updates of all transactions that commit before it started. As a result, T_i effectively sees a snapshot of the database as of the time it started.³

²Many implementations create versions even before the transaction starts validation; since the version timestamp is not available at this point, the timestamp is set to infinity initially, and is updated to the correct value at the time of validation. Further optimizations are used in actual implementations, but we ignore them for simplicity.

³To efficiently find the correct version of a data item for a given timestamp, many implementations store not only the timestamp when a version was created, but also the timestamp when the next version was created, which can be considered an *invalidation timestamp* for that version; the version is valid between the creation and invalidation timestamps. The current version of a data item has the invalidation timestamp set to infinity.

18.8.2 Validation Steps for Update Transactions

Deciding whether or not to allow an update transaction to commit requires some care. Potentially, two transactions running concurrently might both update the same data item. Since these two transactions operate in isolation using their own private snapshots, neither transaction sees the update made by the other. If both transactions are allowed to write to the database, the first update written will be overwritten by the second. The result is a **lost update**. This must be prevented. There are two variants of snapshot isolation, both of which prevent lost updates. They are called *first committer wins* and *first updater wins*. Both approaches are based on testing the transaction against concurrent transactions.

A transaction T_j is said to be **concurrent with** a given transaction T_i if it was active or partially committed at any point from the start of T up to the point when validation of T_i started. Formally, T_j is concurrent with T_i if either

$$\begin{aligned} & \text{StartTS}(T_j) \leq \text{StartTS}(T_i) \leq \text{CommitTS}(T_j), \text{ or} \\ & \text{StartTS}(T_i) \leq \text{StartTS}(T_j) \leq \text{CommitTS}(T_i). \end{aligned}$$

Under **first committer wins**, when a transaction T_i starts validation, the following actions are performed as part of validation, after its *CommitTS* is assigned. (We assume for simplicity that only one transaction performs validation at a time, although real implementations do support concurrent validation.)

- A test is made to see if any transaction that was concurrent with T has already written an update to the database for some data item that T intends to write. This can be done by checking for each data item d that T_i intends to write, whether there is a version of the data item d whose timestamp is between $\text{StartTS}(T_i)$ and $\text{CommitTS}(T_i)$.⁴
- If any such data item is found, then T_i aborts.
- If no such data item is found, then T commits and its updates are written to the database.

This approach is called “first committer wins” because if transactions conflict, the first one to be tested using the above rule succeeds in writing its updates, while the subsequent ones are forced to abort. Details of how to implement these tests are addressed in Exercise 18.15.

Under **first updater wins**, the system uses a locking mechanism that applies only to updates (reads are unaffected by this, since they do not obtain locks). When a transaction T_i attempts to update a data item, it requests a *write lock* on that data item. If the lock is not held by a concurrent transaction, the following steps are taken after the lock is acquired:

- If the item has been updated by any concurrent transaction, then T_i aborts.

⁴There are alternative implementations, based on keeping track of read and write sets for transactions.

- Otherwise T_i may proceed with its execution, including possibly committing.

If, however, some other concurrent transaction T_j already holds a write lock on that data item, then T_i cannot proceed, and the following rules are followed:

- T_i waits until T_j aborts or commits.
 - If T_j aborts, then the lock is released and T_i can obtain the lock. After the lock is acquired, the check for an update by a concurrent transaction is performed as described earlier: T_i aborts if a concurrent transaction had updated the data item, and it proceeds with its execution otherwise.
 - If T_j commits, then T_i must abort.

Locks are released when the transaction commits or aborts.

This approach is called “first updater wins” because if transactions conflict, the first one to obtain the lock is the one that is permitted to commit and perform its update. Those that attempt the update later abort unless the first updater subsequently aborts for some other reason. (As an alternative to waiting to see if the first updater T_j aborts, a subsequent updater T_i can be aborted as soon as it finds that the write lock it wishes to obtain is held by T_j .)

18.8.3 Serializability Issues and Solutions

Snapshot isolation is attractive in practice because transactions that read a lot of data (typically for data analysis) do not interfere with shorter update transactions (typically used for transaction processing). With two-phase locking, such long read-only transactions would block update transactions for long periods of time, which is often unacceptable.

It is worth noting that integrity constraints that are enforced by the database, such as primary-key and foreign-key constraints, cannot be checked on a snapshot; otherwise it would be possible for two concurrent transactions to insert two tuples with the same primary key value, or for a transaction to insert a foreign key value that is concurrently deleted from the referenced table. This problem is handled by checking these constraints on the current state of the database, rather than on the snapshot, as part of validation at the time of commit.

Even with the above fix, there is still a serious problem with the snapshot isolation scheme as we have presented it and as it is implemented in practice: *snapshot isolation does not ensure serializability!*

Next we give examples of possible nonserializable executions under snapshot isolation. We then outline the *serializable snapshot isolation* technique that is supported by some databases, which extends the snapshot isolation technique to ensure serializability. Snapshot isolation implementations that do not support serializable snapshot isolation often support SQL extensions that allow the programmer to ensure serializability even with snapshot isolation; we study these extensions at the end of the section.

T_i	T_j
read(A)	
read(B)	
	read(A)
	read(B)
$A=B$	
	$B=A$
write(A)	
	write(B)

Figure 18.20 Nonserializable schedule under snapshot isolation.

- Consider the transaction schedule shown in Figure 18.20. Two concurrent transactions T_i and T_j both read data items A and B . T_i sets $A = B$ and writes A , while T_j sets $B = A$ and writes B . Since T_i and T_j are concurrent, under snapshot isolation neither transaction sees the update by the other in its snapshot. But, since they update different data items, both are allowed to commit regardless of whether the system uses the first-update-wins policy or the first-committer-wins policy.

However, the execution is not serializable, since it results in swapping of the values of A and B , whereas any serializable schedule would set both A and B to the same value: either the initial value of A or the initial value of B , depending on the order of T_i and T_j .

It can be easily seen that the precedence graph has a cycle. There is an edge in the precedence graph from T_i to T_j because T_i reads the value of A that existed before T_j writes A . There is also an edge in the precedence graph from T_j to T_i because T_j reads the value of B that existed before T_i writes B . Since there is a cycle in the precedence graph, the result is a nonserializable schedule.

This situation, where each of a pair of transactions has read a data item that is written by the other, but the set of data items written by the two transactions do not have any data item in common, is referred to as **write skew**.

- As another example of write skew, consider a banking scenario. Suppose that the bank enforces the integrity constraint that the sum of the balances in the checking and the savings account of a customer must not be negative. Suppose the checking and savings balances for a customer are \$100 and \$200, respectively. Suppose that transaction T_{36} withdraws \$200 from the checking account, after verifying the integrity constraint by reading both balances. Suppose that concurrently transaction T_{37} withdraws \$200 from the savings account, again after verifying the integrity constraint. Since each of the transactions checks the integrity constraint on its own snapshot, if they run concurrently each will believe that the sum of the balances after the withdrawal is \$100, and therefore its withdrawal does not violate

the constraint. Since the two transactions update different data items, they do not have any update conflict, and under snapshot isolation both of them can commit.

Unfortunately, in the final state after both T_{36} and T_{37} have committed, the sum of the balances is \$100, violating the integrity constraint. Such a violation could never have occurred in any serial execution of T_{36} and T_{37} .

- Many financial applications create consecutive sequence numbers, for example to number bills, by taking the maximum current bill number and adding 1 to the value to get a new bill number. If two such transactions run concurrently, each would see the same set of bills in its snapshot, and each would create a new bill with the same number. Both transactions pass the validation tests for snapshot isolation, since they do not update any tuple in common. However, the execution is not serializable; the resultant database state cannot be obtained by any serial execution of the two transactions. Creating two bills with the same number could have serious legal implications.

The above problem is in fact an example of the phantom phenomenon, which we saw in Section 18.4.3, since the insert performed by each transaction conflicts with the read performed by the other transaction to find the maximum bill number, but the conflict is not detected by snapshot isolation.⁵

The problems listed above seem to indicate that the snapshot isolation technique is vulnerable to many serializability problems and should never be used. However, serializability problems are relatively rare for two reasons:

1. The fact that the database must check integrity constraints at the time of commit, and not on a snapshot, helps avoid inconsistencies in many situations. For example, in the financial application example that we saw earlier, the bill number would likely have been declared as a primary key. The database system would detect the primary key violation outside the snapshot and roll back one of the two transactions.

It was shown that primary key constraints ensured that all transactions in a popular transaction processing benchmark, TPC-C, were free from nonserializability problems, when executed under snapshot isolation. This was viewed as an indication that such problems are rare. However, they do occur occasionally, and when they occur they must be dealt with.⁶

2. In many applications that are vulnerable to serializability problems, such as skew writes, on some data items, the transactions conflict on other data items, ensuring

⁵The SQL standard uses the term *phantom problem* to refer to nonrepeatable predicate reads, leading some to claim that snapshot isolation avoids the phantom problem; however, such a claim is not valid under our definition of phantom conflict.

⁶For example, the problem of duplicate bill numbers actually occurred several times in a financial application in I.I.T. Bombay, where (for reasons too complex to discuss here) the bill number was not a primary key, and it was detected by financial auditors.

such transactions cannot execute concurrently; as a result, the execution of such transactions under snapshot isolation remains serializable.

Nonserializable may nevertheless occur with snapshot isolation. The impact of nonserializable execution due to snapshot isolation is not very severe for many applications. For example, consider a university application that implements enrollment limits for a course by counting the current enrollment before allowing registration. Snapshot isolation could allow the class enrollment limit to be exceeded. However, this may happen very rarely, and if it does, having one extra student in a class is usually not a major problem. The fact that snapshot isolation allows long read transactions to execute without blocking updaters is a large enough benefit for many such applications to live with occasional glitches.

Nonserializability may not be acceptable for many other applications, such as financial applications. There are several possible solutions.

- A modified form of snapshot isolation, called serializable snapshot isolation, can be used if it is supported by the database system. This technique extends the snapshot isolation technique in a way that ensures serializability.
- Some systems allow different transactions to run under different isolation levels, which can be used to avoid the serializability problems mentioned above.
- Some systems that support snapshot isolation provide a way for SQL programmers to create artificial conflicts, using a **for update** clause in SQL, which can be used to ensure serializability.

We briefly outline each of these solutions below.

Since version 9.1, PostgreSQL implements a technique called serializable snapshot isolation, which ensures serializability; in addition, PostgreSQL versions from 9.1 onwards include an index-locking-based technique to provide protection against phantom problems.

The intuition behind the **serializable snapshot isolation (SSI)** protocol is as follows: Suppose we track all conflicts (i.e., write-write, read-write, and write-read conflicts) between transactions. Recall from Section 17.6 that we can construct a transaction precedence graph which has a directed edge from T_1 to T_2 if transactions T_1 and T_2 have conflicting operations on a tuple, with T_1 's action preceding T_2 's action. As we saw in Section 17.6, one way to ensure serializability is to look for cycles in the transaction precedence graph and roll back transactions if a cycle is found.

The key reason for loss of serializability with snapshot isolation is that read-write conflicts, where a transaction T_1 writes a version of an object, and a transaction T_2 subsequently reads an earlier version of the object, are not tracked by snapshot isolation. This conflict can be represented by a read-write conflict edge from T_2 to T_1 .

It has been shown that in all cases where snapshot isolation allows nonserializable schedules, there must be a transaction that has both an incoming read-write conflict

edge and an outgoing read-write conflict edge (all other cases of cycles in the conflict graph are caught by the snapshot isolation rules). Thus, serializable snapshot isolation implementations track all read-write conflicts between concurrent transactions to detect if a transaction has both an incoming and an outgoing read-write conflict edge. If such a situation is detected, one of the transactions involved in the read-write conflicts is rolled back. This check is significantly cheaper than tracking all conflicts and looking for cycles, although it may result in some unnecessary rollbacks.

It is also worth mentioning that the technique used by PostgreSQL to prevent phantoms uses index locking, but the locks are not held in a two-phase manner. Instead, they are used to detect potential conflicts between concurrent transactions and must be retained for some time even after a transaction commits, to allow checks against other concurrent transactions. The index-locking technique used by PostgreSQL also does not result in any deadlocks.

SQL Server offers the option of allowing some transactions to run under *snapshot* isolation, while allowing others to run under the *serializable* isolation level. Running long read-only transactions under the snapshot isolation level while running update transactions under the serializable isolation level ensures that the read-only transaction does not block updaters, while also ensuring that the above anomalies cannot occur.

In Oracle versions till at least Oracle 12c (to the best of our knowledge), and in PostgreSQL versions prior to 9.1, the *serializable* isolation level actually implements snapshot isolation. As a result, even with the isolation level set to serializable, it is possible that the database permits some schedules that are not serializable.

If an application has to run under snapshot isolation, on several of these databases an application developer can guard against certain snapshot anomalies by appending a **for update** clause to the SQL select query as illustrated below:

```
select *
from instructor
where ID = 22222
for update;
```

Adding the **for update** clause causes the system to treat data that are read as if they had been updated for purposes of concurrency control. In our first example of write skew shown in Figure 18.20, if the **for update** clause were appended to the select queries that read the values of *A* and *B*, only one of the two concurrent transactions would be allowed to commit since it appears that both transactions have updated both *A* and *B*.

Formal methods exist (see the online bibliographical notes) to determine whether a given mix of transactions runs the risk of nonserializable execution under snapshot isolation and to decide on what conflicts to introduce (using the **for update** clause, for example) to ensure serializability. Such methods can work only if we know in advance what transactions are being executed. In some applications, all transactions are from a predetermined set of transactions, making this analysis possible. However, if the application allows unrestricted, ad hoc transactions, then no such analysis is possible.